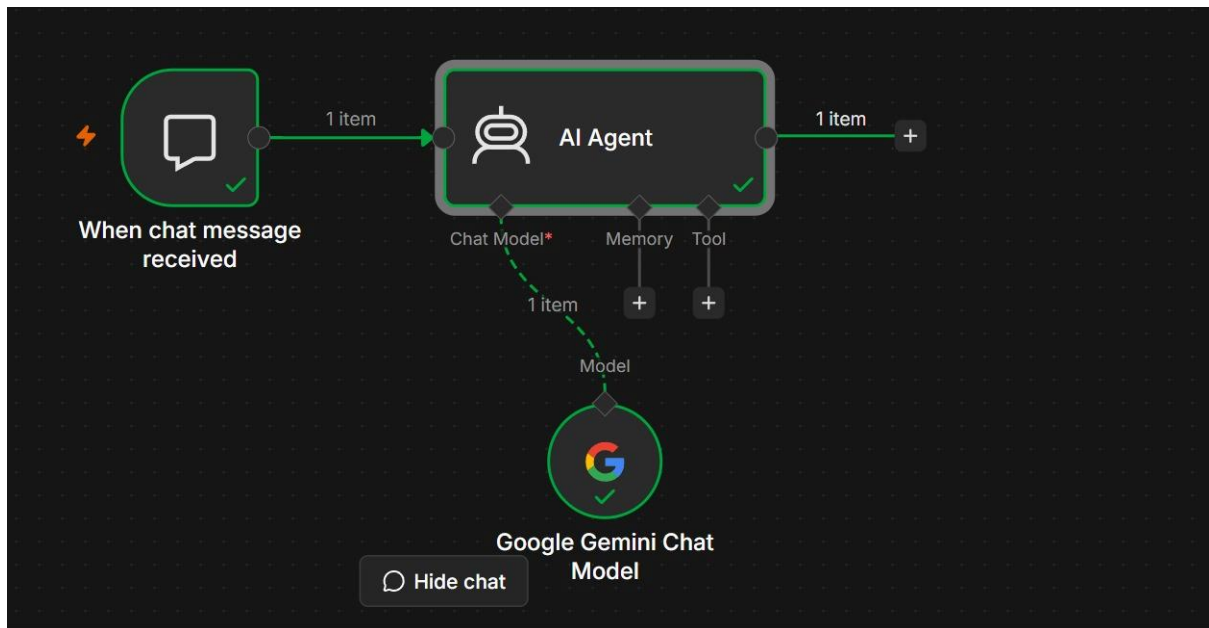


## PROJECT-29

### AI JavaScript Helper



### Project Objective

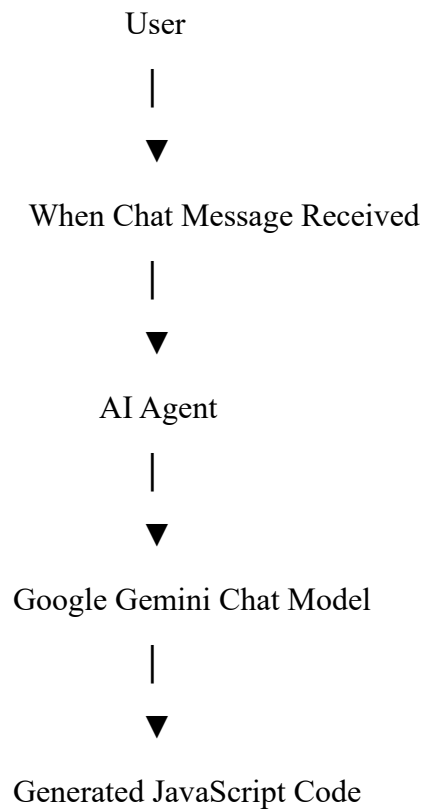
The objective of this project is to build an **AI-powered JavaScript Helper** that assists developers by generating, explaining, debugging, optimizing, and improving JavaScript code based on natural language requests. The AI Agent understands the user's programming requirements and provides accurate JavaScript solutions with explanations.

The AI Agent can help with:

- JavaScript code generation
- Debugging JavaScript code
- Explaining JavaScript concepts
- DOM manipulation
- ES6+ features
- Async/Await programming
- API integration
- Array and Object operations
- Event handling
- Code optimization

---

### Workflow Architecture



---

## Workflow Components

| Step | Node                       | Purpose                                                |
|------|----------------------------|--------------------------------------------------------|
| 1    | When Chat Message Received | Receives the user's JavaScript request                 |
| 2    | AI Agent                   | Understands the coding request and prepares the prompt |
| 3    | Google Gemini Chat Model   | Generates or explains JavaScript code                  |

---

## Prerequisites

Before building this project, ensure you have:

- n8n (Cloud or Self-hosted)
  - Google AI Studio API Key
  - Google Gemini Chat Model configured
  - Internet connection
- 

## Step 1 – Create Workflow

Open **n8n**

Click

**New Workflow**

Rename it

**AI JavaScript Helper**

---

## **Step 2 – Add Chat Trigger**

Add

**When Chat Message Received**

### **Purpose**

Receives the user's JavaScript coding request.

### **Example Input**

Write a JavaScript function that removes duplicate values from an array.

### **Output**

```
{  
  "chatInput": "Write a JavaScript function that removes duplicate values from an array."  
}
```

---

## **Step 3 – Add AI Agent**

Connect

When Chat Message Received

|



AI Agent

### **Configure**

#### **Source for Prompt**

Connected Chat Trigger Node

#### **Prompt**

```
{{ $json.chatInput }}
```

---

## Step 4 – Configure the System Prompt

Paste the following into the **System Message**.

You are an AI JavaScript Helper.

Your task is to assist users with JavaScript programming by generating, debugging, explaining, and optimizing code.

Instructions:

- Carefully understand the user's programming request.
- Generate clean, efficient, and well-formatted JavaScript code.
- Follow modern JavaScript (ES6+) best practices whenever possible.
- Include comments in the code where appropriate.
- Briefly explain how the code works.
- If the user provides existing code, identify errors and suggest improvements.
- Recommend better approaches when applicable.
- Return only the JavaScript solution and its explanation.

---

## Step 5 – Add Google Gemini Chat Model

Click

+ **Chat Model**

Choose

**Google Gemini Chat Model**

**Configuration**

| Setting  | Value                                                |
|----------|------------------------------------------------------|
| Provider | Google Gemini                                        |
| Model    | Gemini 2.5 Flash (or another available Gemini model) |

| Setting | Value |
|---------|-------|
|---------|-------|

|                |                          |
|----------------|--------------------------|
| Authentication | Google AI Studio API Key |
|----------------|--------------------------|

Connect it to the **AI Agent**.

---

### Final Workflow

When Chat Message Received

|



AI Agent

|



Google Gemini Chat Model

---

### Workflow Execution

#### Step 1

User enters

**Write a JavaScript function to reverse a string.**



#### Step 2

The **When Chat Message Received** node receives the user's request.



#### Step 3

The **AI Agent** analyzes the programming requirement and prepares the prompt.



#### Step 4

The prompt is sent to the **Google Gemini Chat Model**.



#### Step 5

Gemini generates JavaScript code with a brief explanation.

↓

## Step 6

The generated JavaScript solution is returned to the user.

---

### Sample Inputs and Outputs

#### Example 1

##### Input

Write a JavaScript function to reverse a string.

##### Output

```
function reverseString(str) {  
    return str.split("").reverse().join("");  
}
```

```
console.log(reverseString("Hello"));
```

##### Explanation

This function splits the string into an array, reverses the array, and joins it back into a string.

---

#### Example 2

##### Input

Write a JavaScript function to remove duplicate values from an array.

##### Output

```
function removeDuplicates(arr) {  
    return [...new Set(arr)];  
}
```

```
console.log(removeDuplicates([1,2,2,3,4,4,5]));
```

##### Explanation

The Set object stores only unique values, and the spread operator converts it back into an array.

---

### Example 3

#### Input

Explain the difference between let, const, and var.

#### Output

##### var

- Function scoped
- Can be redeclared
- Can be updated

##### let

- Block scoped
- Cannot be redeclared
- Can be updated

##### const

- Block scoped
  - Cannot be redeclared
  - Cannot be reassigned
- 

### Example 4

#### Input

Write JavaScript code to fetch data from a REST API.

#### Output

```
async function fetchUsers() {  
    const response = await fetch("https://api.example.com/users");  
    const data = await response.json();  
    console.log(data);  
}
```

```
fetchUsers();
```

## Explanation

This example uses the Fetch API with async/await to retrieve JSON data from a REST API.

---

## Workflow Diagram

User

|



When Chat Message Received

|



AI Agent

|



Google Gemini Chat Model

|



Generated JavaScript Solution

---

## Error Handling

### Error: No Prompt Specified

#### Cause

The AI Agent did not receive the user's request.

#### Solution

Verify that the **Prompt** field contains:

```
{{ $json.chatInput }}
```

---

### Error: Authentication Failed

#### Cause

Invalid Google Gemini API Key.

## **Solution**

Reconnect or update your Google AI Studio credentials.

---

## **Error: Too Many Requests**

### **Cause**

Gemini API rate limit exceeded.

### **Solution**

- Wait a few minutes.
  - Check your API quota.
  - Retry the request.
- 

## **Learning Outcomes**

After completing this project, you will be able to:

- Build an AI-powered JavaScript Helper in n8n.
  - Configure an AI Agent with Google Gemini.
  - Design prompts for JavaScript code generation and debugging.
  - Generate JavaScript solutions from natural language.
  - Build conversational AI workflows for software development.
- 

## **Real-World Applications**

- JavaScript code generation
- Code debugging
- Web development assistance
- Front-end development
- API integration support
- Learning JavaScript
- Developer productivity tools
- Coding interview preparation
- Technical education

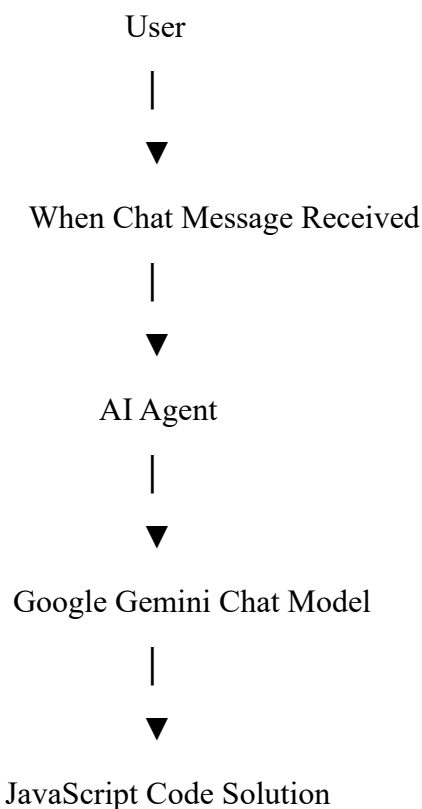
- AI-powered programming assistants
- 

## Practice Challenge

Enhance this project by adding:

1. A programming language selector (JavaScript, TypeScript, Node.js).
  2. A code explanation mode for beginners.
  3. A bug-fixing mode for existing JavaScript code.
  4. A Memory node to remember the user's preferred coding style.
  5. A Google Sheets node to save generated JavaScript snippets.
  6. An HTTP Request node or Code Execution node to automatically test the generated JavaScript code.
- 

## Final Workflow



## **Expected Result**

The **AI JavaScript Helper** accepts a JavaScript programming request from the user, uses the **Google Gemini Chat Model** to generate, explain, debug, or optimize JavaScript code, and returns a clean, well-structured solution with a brief explanation. This project teaches prompt engineering, AI-assisted programming, and conversational workflow design in n8n, making it an excellent foundation for building intelligent coding assistants and software development automation solutions.